

월간 국방과 기술

미래 IT 10대 전략기술 ‘AI Driven Development’



작성자 : 권태욱, 윤지영(203.255.xxx.xxx)

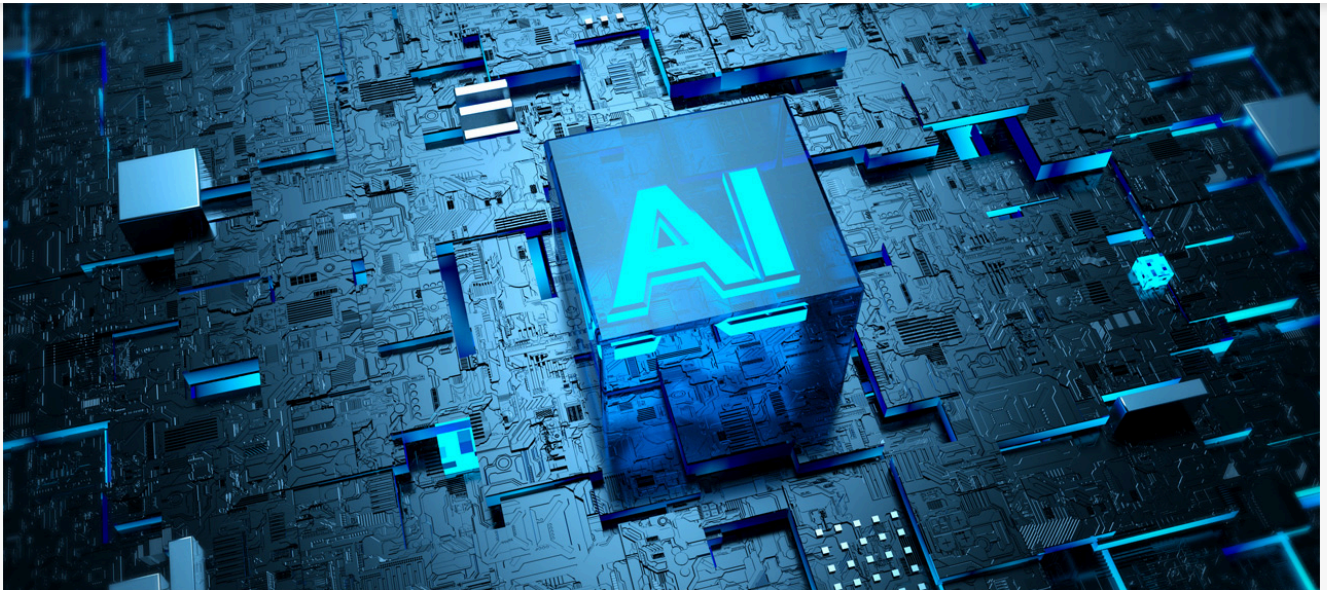
입력 2019-12-10 17:04:48

조회수 3662 | 댓글 0 | 추천 0

미래 IT 10대 전략기술 ‘AI Driven Development’

권태욱 국방대학교 컴퓨터공학과 교수

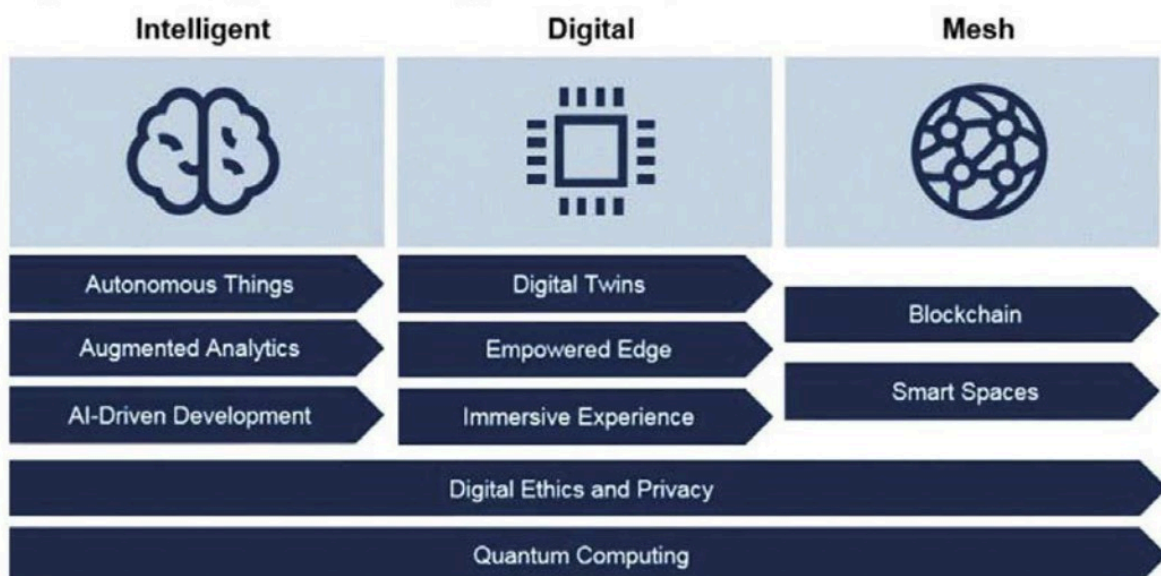
윤지영 국방대학교 국방관리대학원 컴퓨터공학 석사과정 공군 소령



• AI Driven Development의 개념

미국의 정보기술 연구 및 자문회사인 ‘가트너’는 매년 주목해야 할 10대 전략기술 트렌드를 선정하는데, [그림 1]과 같이 2019년 10대 전략 기술로 AI Driven Development(AI 주도 개발)를 선정하였다.

Top 10 Strategic Technology Trends for 2019



ID: 374252

© 2018 Gartner, Inc.

AI Driven Development란 소프트웨어를 개발하는 과정에서 기존의 소프트웨어 개발자가 모든 것을 개발하는 대신 AI 기능이 제공되는 개발도구를 이용하여 솔루션 또는 애플리케이션을 개발하는 것을 의미한다. AI Driven Development가 상용화된다면 비전문가도 AI 구동 틀을 활용하여 자동으로 새로운 솔루션을 생성할 수 있게 되어 ‘시민 애플리케이션 개발자’의 시대가 열릴 것으로 기대된다.

• AI(Artificial Intelligence)

AI란 컴퓨터가 인간의 지능 활동을 모방할 수 있도록 하는 것으로 인간의 지능이 할 수 있는 사고·학습·모방·자기 계발 등을 컴퓨터가 스스로 할 수 있도록 연구하는 것이다. 초기의 인공지능은 게임·바둑 등의 분야에서만 사용되는 정도였으나, 실생활에 응용되기 시작하면서 지능형 로봇 등 활용 분야가 비약적으로 발전하게 되었다.

신경망, 퍼지이론, 패턴인식, 전문가 시스템, 자연어 인식, 이미지 처리, 로봇 공학 등 다양한 분야에서 연구가 이루어지고 있으며, 특히 현대에는 정보기술의 여러 분야에서 인공지능적 요소를 도입하여 문제 해결에 활용하기 위해 연구되고 있다.

사실 AI의 개념은 이미 1950년대부터 거론되어 왔는데, 1950년 앨런 튜링은 ‘계산기계와 지성’이라는 논문을 통해 ‘기계가 사람처럼 생각할 수 있다’면서 컴퓨터와 대화를 나누어 컴퓨터의 반응을 인간과 구별할 수 없다면, 컴퓨터가 생각할 수 있는 것으로 간주해야 한다고 주장하였다.

1955년 존 매카시의 ‘지능이 있는 기계를 만들기 위한 과학과 공학’이라는 논문에서 인공지능이라는 단어가 처음 등장하였고, 1956년에 열린 다스머스 학회에서 공식적으로 ‘인공지능’이라는 용어를 사용하였던 것처럼 컴퓨터가 사람과 같이 생각하고 학습하여 자기 계발하고자 하는 바램은 우리가 생각하는 것보다 꽤 오래전부터 시작되었다고 할 수 있다.

• Software Development란?

Software Development란 시장의 목표나 사용자의 요구를 소프트웨어 제품으로 만드는 과정을 말한다. 좀 더 자세히 들어가 보면 응용프로그램, 프레임워크 또는 기타 소프트웨어 구성요소를 만들고 유지하는데 필요한 착안, 요구사항 정의, 설계, 프로그래밍, 문서화, 테스트, 버그 수정 등의 과정을 말한다.

Software Development를 좁은 의미로 보자면 소스코드를 작성하고 유지하는 과정이라 할 수 있으며, 넓은 의미로 본다면 원하는 소프트웨어의 개념에서 운영 유지보수까지 계획되고 구조화된 과정에 관여하는 모든 것을 뜻한다. 오늘날 소프트웨어가 다양해지고 복잡해짐에 따라 개발 및 구입비용이 크게 증가하게 되는데, 실제 프로그램 개발에 필요한 비용도 중요하지만, 프로젝트 관리 측면에서 소프트웨어를 개발하고 운영

서 소프트웨어 개발과 유지보수에 대한 체계적이고 합리적인 접근 방법이 필요하게 되었기 때문에 Software Development가 연구되게 된 것이다.

1960년대 초기에는 하드웨어 구매 및 유지보수 비용이 시스템 운영비용의 전부였기 때문에 사실상 소프트웨어 관련 비용이 무시되었으나, 60년대 중반에 들어서면서 컴퓨터 사용자들의 요구사항이 증대되고 소프트웨어 구축비용이 크게 증가하게 되면서 상황이 바뀌게 되었다. 기술 발전으로 하드웨어 가격은 하락한 반면, 소프트웨어 개발 인건비는 계속 상승하였고, 소프트웨어 수요도 늘어나게 되면서 컴퓨터를 효율적으로 사용해야 한다는 인식이 증가하게 된 것이다.

이러한 Software Development는 다음의 3가지 목적이 있는데, **첫째**는 특정 클라이언트 또는 특정 비즈니스를 위한 개발로서 맞춤형 소프트웨어를 예로 들 수 있다. **둘째**는 잠재적인 사용자(고객)로부터 인지된 요구를 충족하기 위해 개발하는 것인데, 우리가 일반적으로 사용하는 상업 및 오픈소스 소프트웨어가 이에 해당된다. 마지막 **세 번째**는 개인이 자신의 개인적인 용도를 위해 개발하는(일상적인 업무를 자동화 하는 등) 목적이 있다.

Software Development의 연구 절차를 살펴보면, **첫 번째**로 요구공학을 들 수 있다. 요구공학 분야는 소프트웨어 개발에서 수행되는 첫 번째 단계로서 개발된 시스템에 대한 고객의 요구를 이해하고, 목표와 제약 사항을 확립하여 시스템을 만족시킬 기능, 성능, 그리고 다른 시스템과의 인터페이스 등을 정의하는 과정이다. 요구공학은 비용 증가, 납기 지연, 품질저하를 방지하기 위한 필수 요건이라 할 수 있다.

두 번째로는 아키텍처 영역이 있다. 아키텍처는 구성 요소와 이 구성 요소들 간의 관계, 시스템의 기능, 속성 및 제약사항 등을 적절히 반영하는 구조가 서로 조직화 된 것이다. 또한 개발하고자 하는 시스템의 전체적인 형태를 표현한 것이라 할 수 있다. 이 아키텍처는 소프트웨어 시스템이 어떻게 구성되어야 하는지와 시스템의 전체 구조 설계를 이해할 수 있도록 도움을 준다.

세 번째는 개발방법론이다. 개발방법론은 시스템을 개발하기 위해 어떠한 방법으로 진행할 것인가를 다루는 분야인데, 대표적인 개발방법론으로는 구조적 방법론, 객체지향 방법론을 예로 들 수 있다. 개발방법론은 개발 기술의 진화에 따라 계속적으로 연구, 발전되고 있으며, 개발조직의 특성 및 여건에 맞게 조정하거나 재정의 될 수 있다. 개발방법론에 대해서는 뒤에서 좀 더 자세히 알아보도록 하겠다.

네 번째 연구 영역으로는 테스트를 들 수 있는데 테스트는 말 그대로 소프트웨어의 결함을 찾아내는 기능을 하기 때문에 프로그램에 포함된 요구, 설계 코딩의 오류를 밝혀내는 것이 테스트의 목적이라 할 수 있다. 테스트 분야는 모듈 단위의 단위 테스트, 시스템 통합 시에 사용되는 통합 테스트, 요구사항을 검증하는 시스템 테스트 등이 있다. 오늘날 소프트웨어의 규모가 커지면서 전체적인 테스트가 불가능하기 때문에 효과적인 테스트를 위한 기법이 활발히 연구되고 있다.

다섯 번째는 프로세스 영역이다. 프로세스란 최종 소프트웨어 제품을 생산하기 위하여 요구되는 인력, 절차, 방법, 장치 및 도구들을 통합하는 수단으로써 프로세스의 정의 방법, 프로세스 관리조직 및 관리기반 구조 등이 주로 연구되고 있다.

여섯 번째는 품질관리 영역인데 여기서 말하는 제품 품질이란 제품 자체가 가지는 품질을 의미하며, 프로세스의 품질이란 소프트웨어를 개발하는 프로세스가 정확하고 우수하면 좋은 소프트웨어를 생산할 가능성이 높다는 것을 의미한다. 이 연구 영역에서는 제품 품질을 보증하기 위한 Software Quality Assurance 활동과 제품 검사, 검토 등을 연구하고 있다.

는 시스템에 대한 지식이나 요구분석 사항, 설계, 문서 등이 포함된다.

재사용 분야에서는 코드 재사용의 한계를 극복하기 위해 코딩 단계 이전의 분석 설계 단계에서 만들어진 산출물을 재사용하려는 노력이 계속되고 있다.

• Software Development 방법론

Software Development 방법론이란 소프트웨어 개발에 필요한 여러 가지 일들의 수행방법과 효율성을 높이기 위해 소프트웨어 공학 원리에 입각한 각종 기법 및 도구를 체계적으로 정리하여 표준화한 것을 말한다. 이는 시스템 구축에 필요한 작업 각각에 대한 상세한 작업수행 절차를 포함하고 있으며, 이러한 작업을 효율적이고 효과적으로 수행하기 위한 공학적 방법이다.

Software Development 방법론은 개발 경험을 축적하고 작업을 표준화하고 모듈화하는 등 재사용을 통한 개발생산성 향상을 위해 필요하다. 또한 효과적인 프로젝트 관리 및 정형화된 절차와 표준 용어의 제공으로 의사소통 수단을 제공해 주며, 각 단계별 검증 및 종결승인을 통한 일정 수준의 품질을 보증해 준다.

소프트웨어 개발에는 다양한 방법론이 있는데, 이러한 프레임워크는 각자의 장·단점을 가지고 진화해왔다고 볼 수 있다. 여기서 중요한 점은 하나의 시스템 개발방법론이 반드시 모든 프로젝트에 적합한 것은 아니며, 다양한 기술, 조직화, 요구사항, 프로젝트와 팀의 고려사항에 따라 다양한 방법론을 적용한다는 것이다.

예를 들어 개발 프로세스를 따라 순차적으로 진행 되는 폭포수 모델의 경우 코딩이 시작되기 전에 대부분의 위험을 평가하고 상세 계획을 세우기 때문에 후반부에 설계 변경이나 재코딩을 최소화할 수 있지만, eXtreme programming의 경우에는 예측성보다는 적응성에 더 높은 가치를 주는 방법이기 때문에 소프트웨어 개발에서 모든 요구사항을 전부 정의해 놓고 진행하는 것이 아니라, 특정 시기에 요구사항이 변경되는 것을 적용하는 방법이다. 이렇듯 각각의 방법론은 저마다의 장·단점을 가지고 있기 때문에 소프트웨어 개발 여건에 맞춰 적합한 방법론을 선택하는 것이 중요하다.

이러한 Software Development 방법론은 일반적으로 어떤 절차를 통해 이루어질까? 대부분의 방법론은 다음의 절차에 따라 소프트웨어를 개발하고 있다.

- ❶ 문제분석
- ❷ 시장조사
- ❸ 제안된 비즈니스 솔루션에 대한 요구사항 수집
- ❹ 소프트웨어 기반 솔루션에 대한 계획 또는 설계
- ❺ 소프트웨어 구현(코딩)
- ❻ 소프트웨어 테스트
- ❼ 서비스 제공
- ❽ 유지보수 및 버그 수정

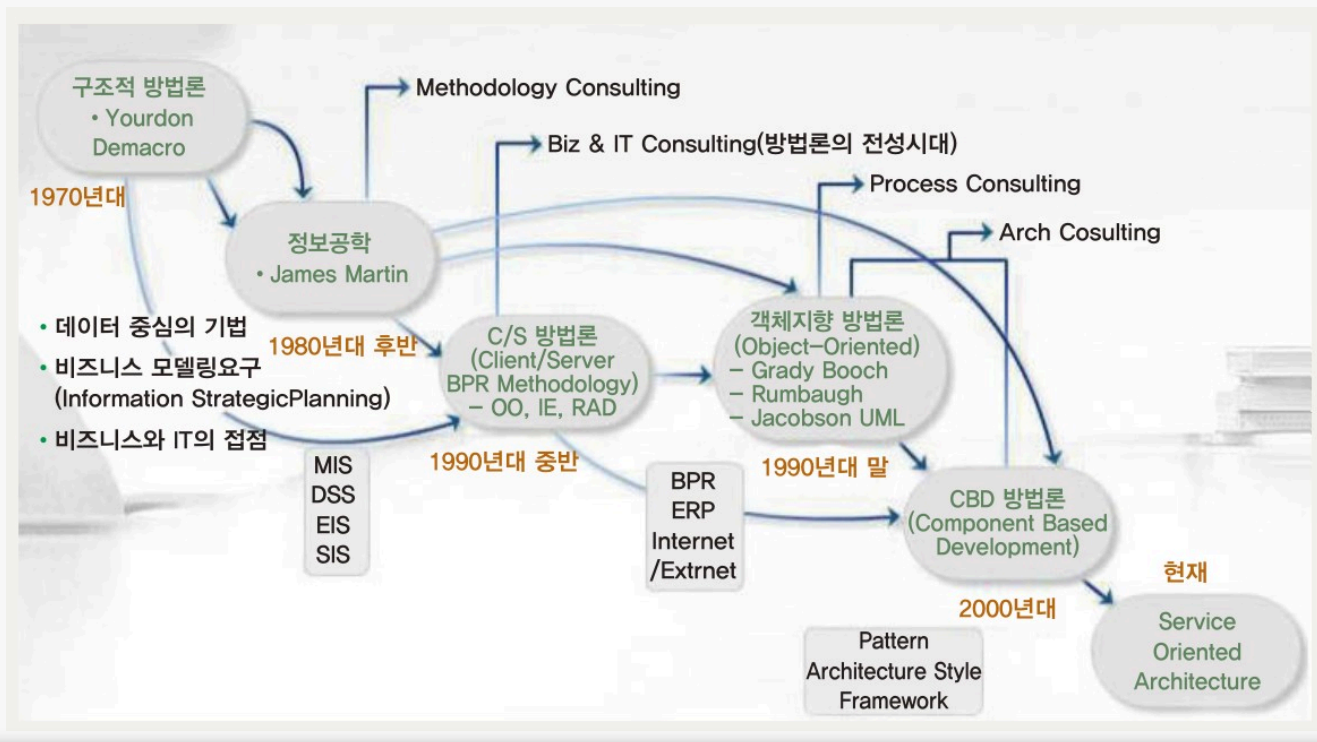
개발 아이디어는 잠재적인 신규 고객, 제품을 거부한 판매 잠재 고객, 내부 소프트웨어 개발 직원, 시장 조사 등에 의해 식별한다. 이 단계에서 경제성이 있다고 확인이 되어야 하며 판매 채널에 적합하고 기존 제품에 대한 영향 등을 평가하고 회사의 마케팅 목표에 적합한지를 고려해야만 한다.

계획·설계 단계(Planning/Designing)에서는 시스템의 모든 구조를 결정하게 된다. 소프트웨어 설계는 프로그램의 데이터 구조, 소프트웨어 구조, 인터페이스 구조, 알고리즘 등을 포함하며 요구사항을 하드웨어와 소프트웨어 시스템으로 나누어 할당하고, 전체 시스템의 아키텍처를 작성하게 된다. 그리고 이 단계를 수행하기 위해서는 개발 언어와 운영체제 및 하드웨어의 구성 요소 등을 모두 알고 있어야만 한다.

다음 **구현·테스트 단계(Implementation/Testing)**에서는 소프트웨어의 설계가 여러 개의 프로그램이나 프로그램 단위로 실체화되게 된다. 소프트웨어 엔지니어가 실제로 프로젝트의 코드를 프로그래밍하는 프로세스의 일부분이라고 할 수 있는데, 다시 말해 설계의 각 부분을 기계가 읽을 수 있는 형태로 변환하는 작업이라고 생각하면 된다. 테스트는 각각의 단위가 주어진 명제를 만족하는지를 검증하는 활동으로서 프로그램이 입력에 따라 요구하는 결과대로 작동하는지 여부와 내부적인 이상 여부 및 오류를 식별하는 단계이다.

마지막으로 **운영·유지보수 단계(Deployment/Main tenance)**는 시스템이 설치되고 실제로 사용되는 단계로서 소프트웨어가 안정적으로 올바르게 사용되어지도록 훈련과 지원을 하는 것이 중요하다. 유지보수는 개발 과정의 이전 단계에서 발견하지 못했던 오류를 수정하고, 새로운 요구사항을 발견하여 시스템 서비스를 향상시키는 것을 뜻하는데 이 운영·유지보수 단계가 소프트웨어 생명 주기에서 가장 활동이 긴 단계라고 할 수 있다.

그렇다면 Software Development 개발방법론에는 어떤 것들이 있을까? 개발방법론은 [그림 2]와 같이 시간이 흐름에 따라 과거의 방법론들을 보완하면서 발전되어 왔다.



[표 1]에서 다양한 Software Development 방법론 중 대표적인 몇 가지를 소개해 보겠다.

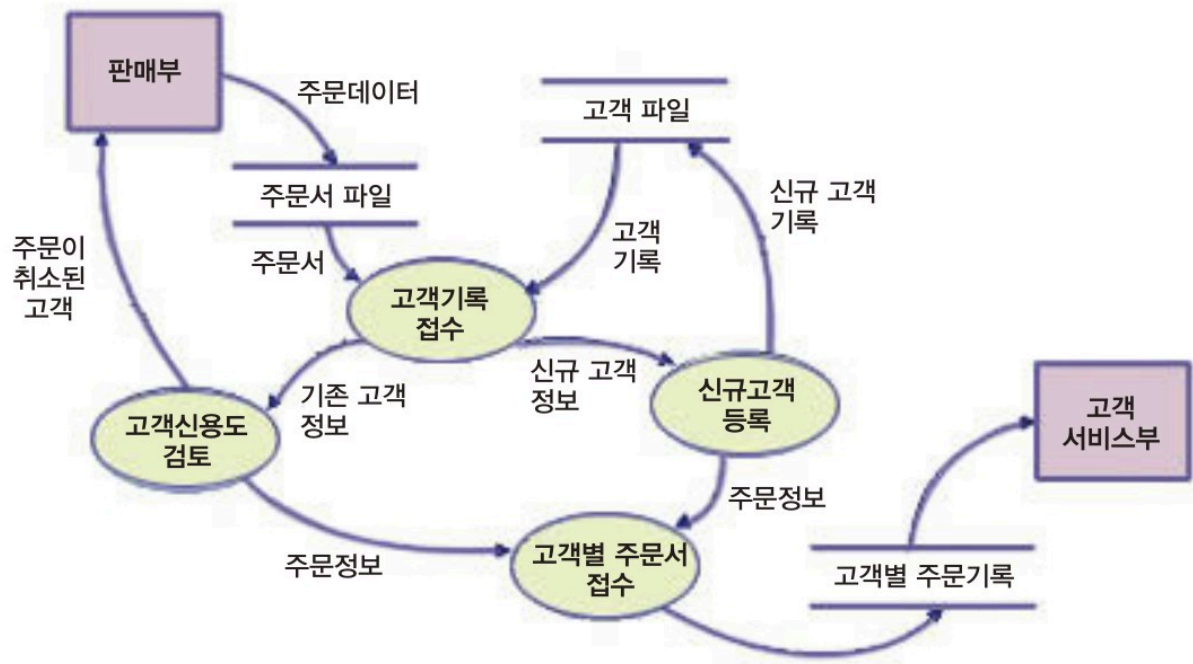
개발방법론	등장시기	패러다임	주요 특징	대표적 방법론
구조적 개발방법론	1970년대	프로세스	프로세스 모델링 중심의 구조적 분석 및 설계	폭포수 모델, 나선형 모델
정보공학 개발방법론	1980년대	데이터	데이터 중심의 분할 정복에 의한 하향식 설계	Information Engineering Methodology
객체지향 개발방법론	1990년대 초반	객체	프로세스와 데이터가 통합된 객체지향 개발방법론 핵심이 프로세스 구조가 아닌 객체구조로 변화	Object Oriented Software Engineering Rational Unified Process
컴포넌트 기반 개발방법론	1990년대 중반	재사용	재사용을 고려한 컴포넌트 기반 개발 방법론	국방 CBD 방법론
제품계열 개발방법론	1990년대 후반	재사용	제품 패밀리 분석을 통한 재사용성 극대화를 위한 핵심자산 중심의 개발방법론	Feature Oriented Reuse Method
애자일 개발방법론	1990년대 후반	시작 적시성	신속한 시장 진입을 목표로 소프트웨어를 개발하는 유연한 개발방법론	eXtreme Programming Open source Software Development
모델 주도 개발방법론	2000년대	모델	모델의 자동화와 변화를 통한 개발 생산성 증대를 위한 개발방법론	Model Based Integration of Embedded System

[표 1] 소프트웨어 개발방법론의 발전 동향

◆ 구조적 개발방법론(Structured programming)

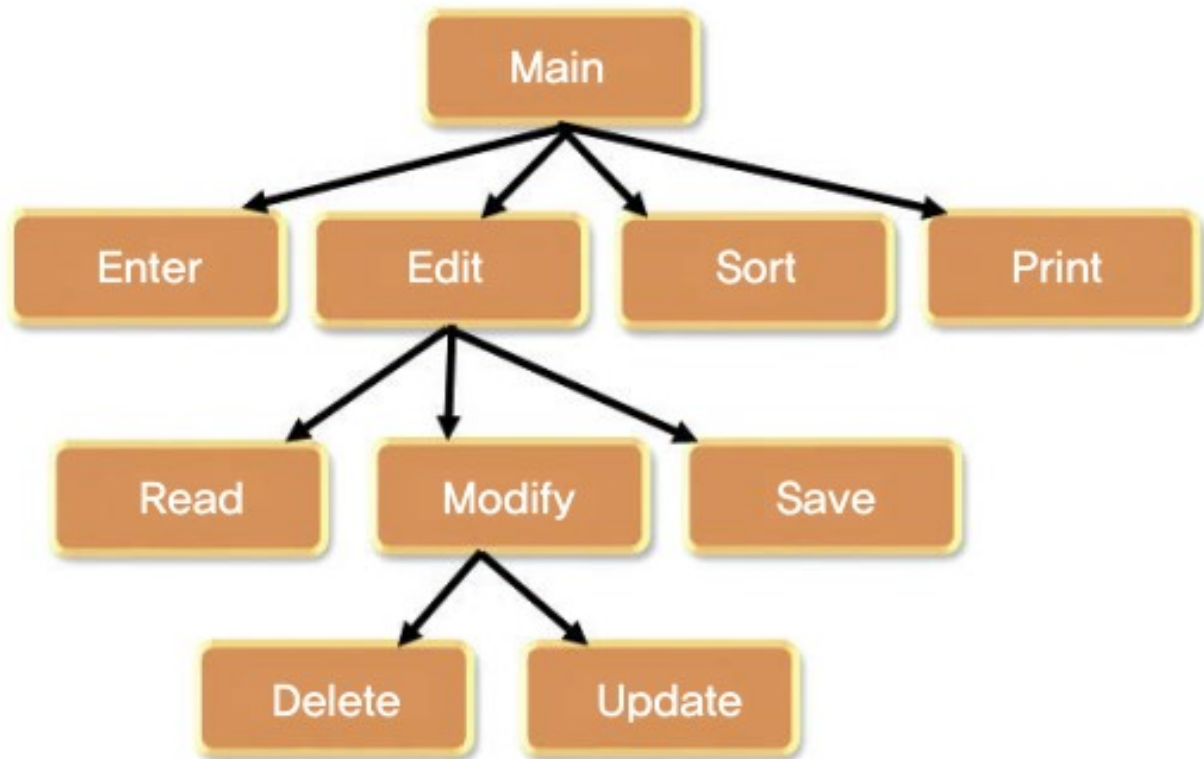
구조적 개발방법론은 업무활동 중심의 방법론이라 할 수 있는데, 정형화된 절차 및 도형 중심의 도구를 이용하여 사용자 요구사항을 파악하고 문서화 한 후 정보와 정보의 구조를 중심으로 분석과 설계, 구현 단계를 거친다. 정형화된 분석 절차에 따라 사용자 요구 사항을 파악하고 도형 중심의 다이어그램을 이용하여 문서화한다.

이러한 구조적 개발방법론Structured programming은 1970년대에 제안되어 80년대에 활발히 사용되는데, 구조적 분석은 실세계의 문제를 처리Process하는 관점으로 모델링한다. 모델링은 [그림 3]과 같이 그래프의 일종인 Data Flow Diagram을 사용하는데, 이 Data Flow Diagram에서 각 노드는 외부 엔티티, 프로세스, 자료 저장소를 표현하며 이들 사이의 관계를 선으로 표시한다. [그림 3]은 Data Flow Diagram의 대표적인 예이다.



[그림 3] Data flow Diagram

구조적 개발방법론Structured programming에서는 복잡한 문제를 쉽게 다루기 위하여 [그림 4]처럼 분할과 정복 원리를 적용하는데, 맨 처음 시스템 전체를 하나의 프로세스로 보고 최상위 레벨에 Data Flow Diagram을 그린다. 복잡한 프로세스가 발생하면 단순한 프로세스로 계속 분할(계층의 말단 프로세스를 구현할 때까지 반복)하여 복잡한 문제를 해결한다.

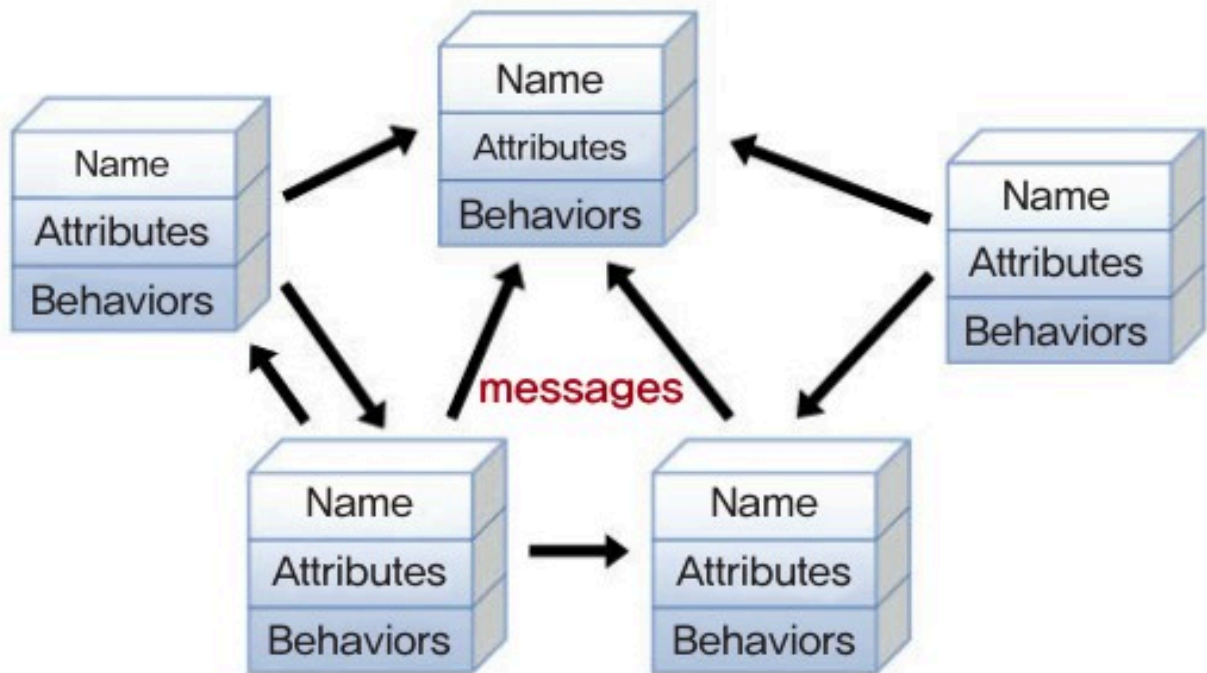


[그림 4] 분할과 정복

쉽게 이해하자면 구조적 설계란 Data Flow Diagram을 구조도로 변경하는 과정이라고 생각하면 되겠다. 그러나 이 구조적 설계방법론 Structured programming은 소프트웨어의 규모와 복잡도가 커지고 Data Flow 위주의 분석과 설계가 한계에 부딪히게 되면서 점차 다른 개발방법론이 요구되게 된다. 그러면서 등장하게 된 것이 바로 객체지향 방법론 Object oriented programming Methodology이다.

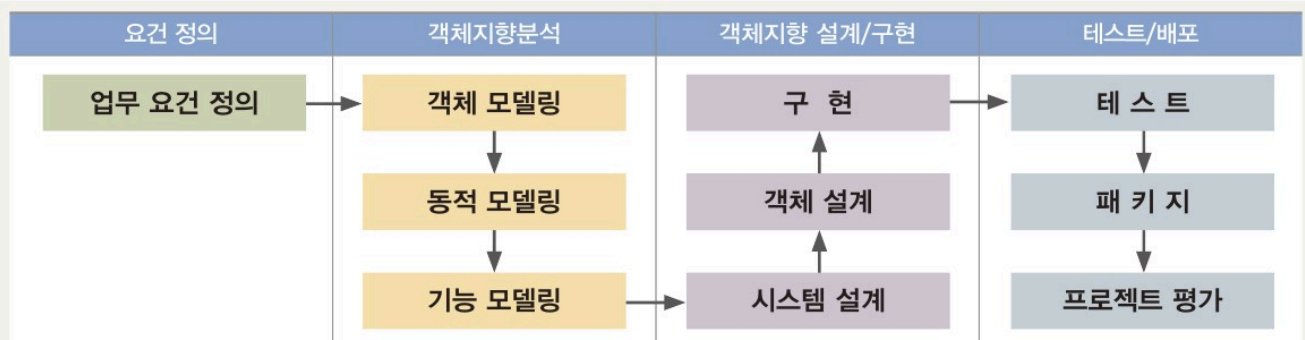
◆ 객체지향 방법론(Object oriented programming Methodology)

객체지향 방법론은 실세계를 객체라는 눈으로 바라보고 객체 사이의 인터랙션을 모델링하는 기법으로서 복잡한 현실 세계를 인간이 이해하는 방식으로 시스템에 적용시키는 개념이며, 이를 위해 [그림 5]와 같이 객체, 클래스, 메시지를 기본 모형으로 제시한다.



[그림 5] 객체, 클래스, 메시지 간의 관계

프로그램을 객체와 객체 간의 인터페이스 형태로 구성하기 위해 문제 영역에서 객체, 클래스 및 이들 간의 관계를 식별해 설계 모델로 변환한다.



[그림 6] 객체지향 개발과정

객체지향 방법론Object oriented programming Methodology의 특징으로는 분석·설계·구현의 벽이 없고 일관성, 재사용성, 유지보수성이 뛰어나며, 전체 공정에서 각 단계 간의 전환과 변경이 자연스럽게 신속하다는 장점이 있다. 또한 현실 세계 및 인간의 사고방식과 유사하기 때문에 요구사항 단계에서 식별된 사용자의 요구사항에 유연하게 대응할 수 있다.

◆ 컴포넌트 기반 방법론(Component Based Development)

컴포넌트 기반 방법론은 재사용 소프트웨어 컴포넌트를 기반으로 소프트웨어 시스템을 개발하는 접근 방법이다. 컴포넌트 기반 방법론Component Based Development은 객체지향 방법론Object oriented programming Methodology이 원래 의도했던 광범위한 재사용을 하지 못했다는 어려움에서 시작되었다. 재사용이 가능한 컴포넌트의 개발 또는 상용 컴포넌트를 조합하여 소프트웨어 개발 생산성과 품질을 높이고 시스템 유지보수 비용을 최소화할 수 있는 개발 방법 프로세스라고 할 수 있다.

컴포넌트 기반 방법론Component Based Development은 시스템을 하나의 일체형으로 구축하는 것이 아니라, 마치 레고 블록처럼 각각의 요소별로 부품화 하여 구축한다. 즉, 기존의 코딩 방식에 의한 개발에서 벗어나 소프트웨어 구성단위Module를 미리 만들어 놓은 뒤 필요한 응용기술을 개발할 때 이 모듈들을 조립하는 방법이라고 생각하면 된다. 그렇기 때문에 컴포넌트 기반 방법론Component Based Development을 사용하면 복잡한 정보 시스템도 신속하게 구축할 수 있다.

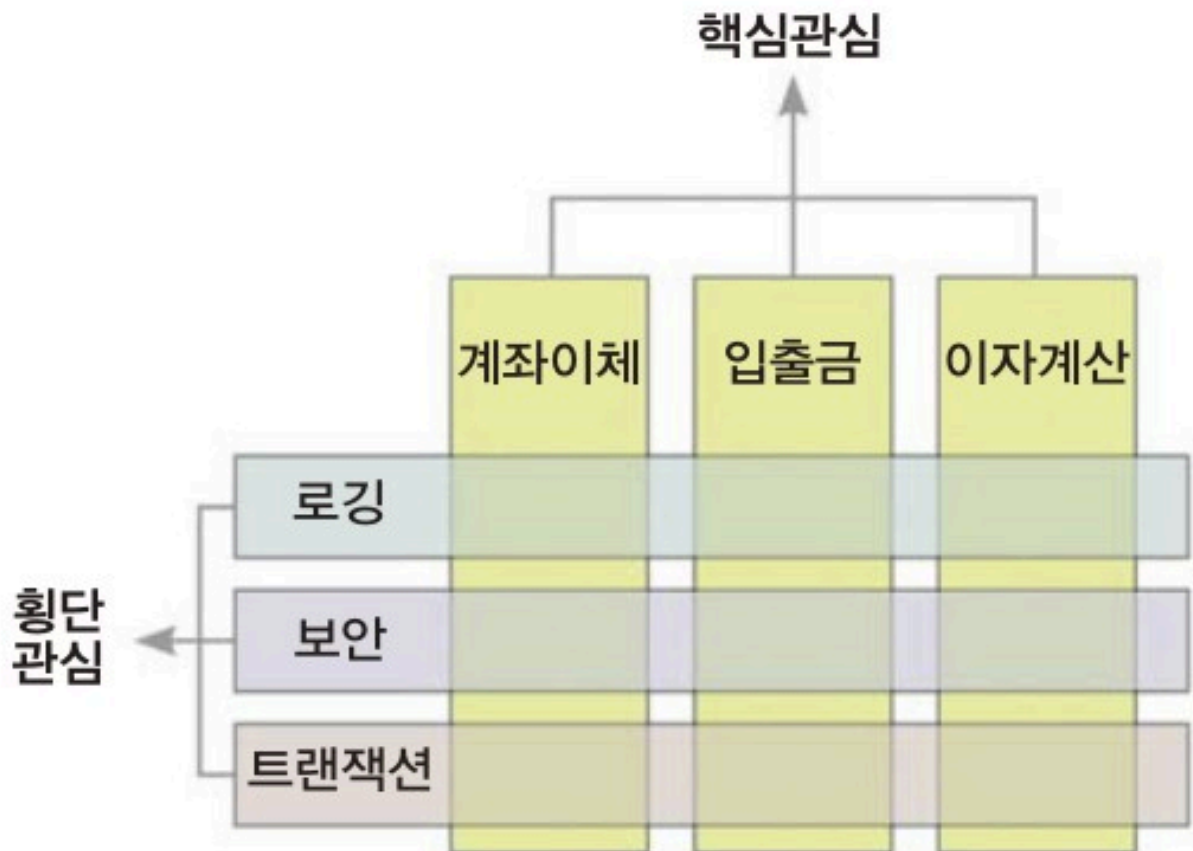
◆ 모델 주도 공학 방법론(Model driven engineering)

모델 주도 공학 방법론은 모든 컴포넌트 기술 요소의 표준 메타모델을 정의하고 이를 기반으로 각 구성 요소를 정의함으로써 호환성 및 시스템간 자동성을 보장하고자 하는 소프트웨어 개발 기술이다.

정형화된 모델을 중심으로 분석, 설계를 수행하고 소스 코드 및 산출물을 자동 생성할 수 있는데, 전통적인 애플리케이션 개발 방식인 코드 중심 개발 방식과 달리 각종 소프트웨어 및 시스템 개발 시 업무 요구 사항과 설계를 시각화하여 제공하므로 기능 구현이 쉽고 시스템을 완성하기 전에 기능 확인과 검증을 효과적으로 수행할 수 있게 해 준다. 그렇기 때문에 프로젝트 진행 결과 전체를 재사용 할 수 있으며, 구현환경과 독립적으로 정의되므로 이식성이 증가하는 장점이 있다.

◆ 관점 지향 프로그래밍(Aspect oriented software development)

소프트웨어 개발의 신속·유연·결함 최소화를 위해 중복 또는 예외적인 영역을 하나로 모아 처리하여 효율성을 높이고 업무별 재코딩하게 되는 것을 지양하는 기법으로 프로그램 내부 여러 곳에서 사용되는 특정 함수에 대한 통제를 중앙집중화 함으로써 개발 절차를 단순화할 수 있게 된다.



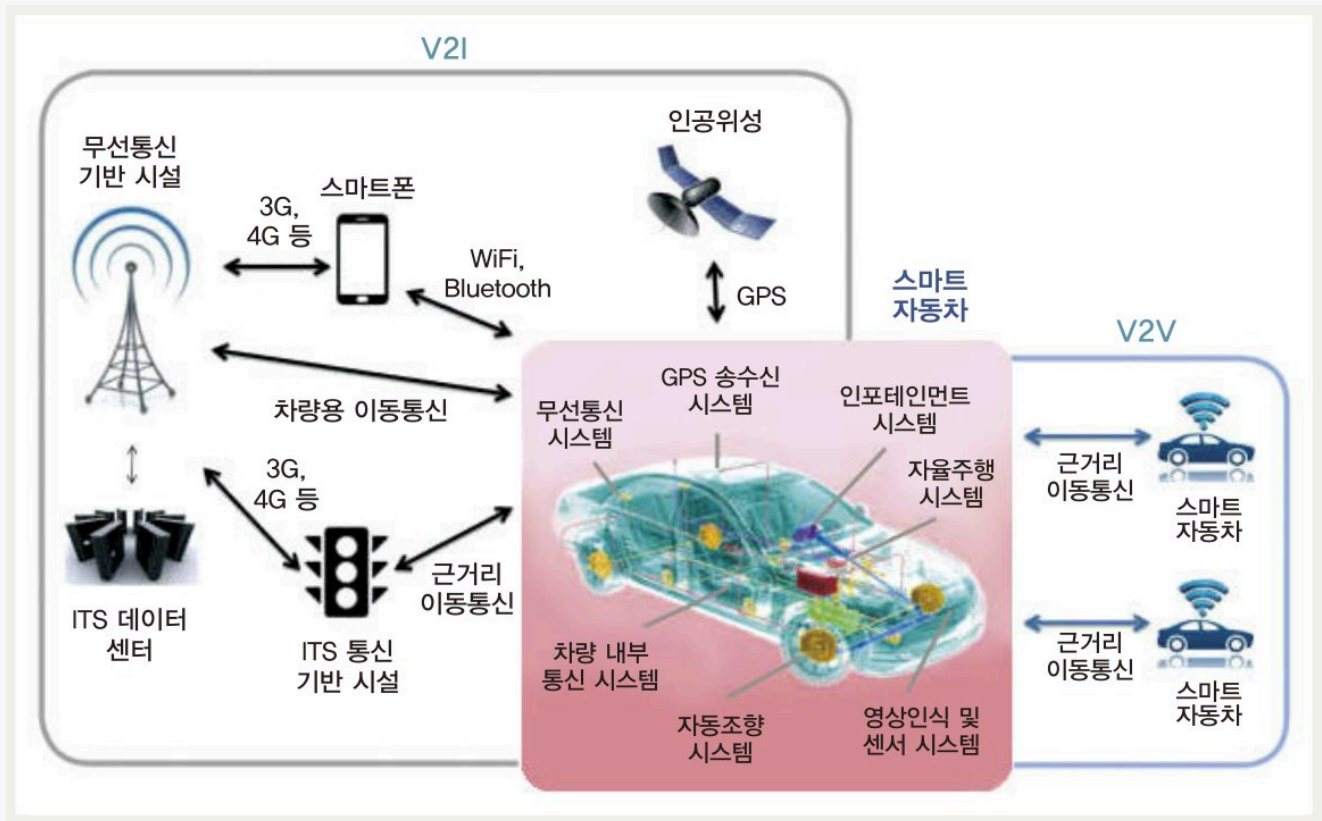
[그림 7] 핵심관심과 횡단관심의 분리

이를 해결하기 위해 관점 지향 프로그래밍 Aspect oriented Software Development에서는 관심을 [그림 7]과 같이 핵심관심과 횡단관심으로 분리하여 기존 클래스의 핵심 로직은 유지하면서 변경은 최소화하여 공통기능의 중복 코드를 재사용할 수 있도록 하는 것이다.

◆ 서비스 지향 모델링(Service oriented modeling)

서비스 지향 모델링이란 대규모 컴퓨터 시스템을 구축할 때의 개념으로 업무상 일처리에 해당하는 소프트웨어 기능을 서비스로 판단하여 그 서비스를 네트워크상에 연동하여 시스템 전체를 구축해 나가는 방법론이다.

정보를 누구나 이용 가능한 서비스로 간주하고 연동과 통합을 전제로 아키텍처를 구축하는데 서로 다른 이



[그림 8] 차량 정보 서비스

서비스 지향 모델링Service Oriented Modeling의 실제 적용사례를 들어보자면 [그림 8]처럼 우리가 항상 이용하는 차량을 예로 들 수 있는데, 차량이 기상, 교통, 지역 시설 정보를 제공하는 다양한 서비스로부터 정보를 통합한 외부 모바일 정보 서비스와 통신하는 것을 생각해 보자. 차량의 정보 시스템이 사용자에게 라디오, 네비게이션, DMB, 커넥티드 장비(블루링크, UVO 등) 등을 통해 GPS 위성, 방송국, 차량제조사 등에서 제공하는 서비스를 전달하는 과정이 바로 서비스 지향 모델링Service Oriented Modeling의 좋은 예라고 할 수 있다.

서비스 지향 모델링Service Oriented Modeling에서는 불필요한 기능은 외부 서비스에 맡기기 때문에 소프트웨어의 소형화가 가능하며, 새로운 서비스를 편리하게 구축할 수 있고, 서비스 사용자는 클라우드 컴퓨팅과 같이 서비스 사용에 따른 비용만 지불하면 되는 등의 장점이 있다.

• AI Driven Development

AI Driven Development란 전문 데이터 과학자들과 전문 개발자들의 협력을 통해 솔루션을 개발해 왔던 지금까지의 방식에서 벗어나, AI(인공지능)를 활용하여 전문 개발자 단독으로 소프트웨어를 개발하는 것을 의미한다. 또한 AI가 개발 과정 자체에 적용되어 다양한 데이터 과학, 소프트웨어 개발 및 테스트 기능을 자동화함으로써 전문적인 소프트웨어 개발에 새로운 기회가 창출될 것으로 기대하게 해 주는 기술이다.

AI Driven Development로 인하여 지금보다 훨씬 편리하고 효율적인 개발 환경을 제공하게 될 것이며, 비전문가도 AI 구동 틀을 활용하게 되면서 ‘시민 애플리케이션 개발’의 시대가 열릴 것으로 예측된다.

NVIDIA의 슈퍼컴퓨터 담당 부사장 짐 맥휴는 인터뷰를 통해 “자가 학습을 통해 파이썬 스크립트를 작성하는 소프트웨어가 나올 것이다. 이렇게 되면 전통적인 코더는 설 자리가 없어질 것이다. 또한 인공지능의 분석 능력으로 막대한 양의 코드를 분석한다면 아마도 알파고처럼 짧은 시간에 급격한 코딩 능력을 보유하게 될 것이며, 인공지능과 딥러닝은 소프트웨어를 개발하는 방법을 근본적으로 바꿔놓을 것”이라고 언급한 바 있다.

소프트웨어 개발 시장은 전문 데이터 개발자가 소프트웨어 개발자와 협력하여 AI 강화 솔루션을 만드는 방식에서 벗어나, 전문 개발자가 서비스로 제공되는 미리 정의된 모델을 사용하여 단독으로 제작하는 방식으로 급속하게 변화하고 있다. 이러한 AI 플랫폼이나 AI 서비스의 진화는 개발자에게 광범위한 AI 강화 솔루션을 제공할 수 있으며, 개발자에게 높은 생산성을 제공하고, 또한 소프트웨어 개발 중 발생하는 낭비와 비효율을 줄일 수 있게 될 것이다.

AI Driven Development를 2019년 10대 전략 기술로 선정한 가트너에서는 AI로 인해 다양한 데이터 과학과 소프트웨어 개발 및 테스트 기능을 자동화하게 되면서 지금까지와는 다른 수준의 기회가 생길 것으로 전망하면서 2019년에는 이러한 기능을 단순화하기 위한 개발 및 테스트들이 시행될 것이며, 2020년에는 가상의 소프트웨어 엔지니어를 활용하여 코드를 생성할 수 있을 것으로 예상하였고, 2022년에는 새로운 소프트웨어 개발팀에 최소 40%는 AI 기반 가상 개발자가 참여할 것으로 예측하였다.

AI Driven Development는 이미 여러 기업에 의해 실현되고 있다. 세계적인 IT 기업인 구글은 2017년 구글 개발자 컨퍼런스에서 ‘과거에는 볼 수 없었던 색다른 소프트웨어 개발방법론을 연구하고 있음’을 발표하면서 인공지능으로 또 다른 인공지능을 만드는 신개념 소프트웨어 개발방법론인 AutoML을 공개하였다. AutoML은 ‘인공지능이 다른 인공지능을 개발한다’는 컨셉을 가지고 머신러닝이 가능한 AI를 구축하였는데, 인공지능이 스스로 신경망 코드와 알고리즘을 구축할 수 있도록 현실화된 것이 바로 AutoML이다.

이는 머신러닝 개발에 필수적인 최적의 설계를 AI에게 맡김으로써 누구나 목표한 인공지능 시스템을 만들 수 있도록 해줄 것으로 예상된다. AutoML은 인공지능 활용 앱에 탑재할 간단한 모델을 몇 분 만에 만들어 낼 수 있으며, 완전한 모델의 경우에도 짧으면 하루 만에 구축이 가능하다고 전해진다. 즉, 머신러닝이나 코딩에 대한 전문적인 지식이 없더라도 누구나 해당 분야의 특정한 필요에 맞게 강력한 머신러닝 모델의 활용을 가능하게 해준다.

또 하나의 거대 IT 기업인 MicroSoft에서도 머신러닝 기반의 자동 코딩 기술이 적용된 DeepCoder를 발표하였는데, DeepCoder는 자동 코딩 기술에 이미 예전부터 연구되어 오던 프로그램 자동생성 기술인 IPS(Inductive Program Synthesis) 기술을 채택하였다.

IPS는 주어진 예제로부터 프로그램 코드를 자동으로 학습하는 기술인데 DeepCoder는 이 IPS에 추가로 프로그램의 특성을 예측하기 위해 신경망을 훈련 하는 방식으로 접근하였다.

용하는 것은 어렵지만 DeepCoder가 상용화된다면 소프트웨어 개발 시장에 큰 파급효과를 불러올 것으로 전망된다.

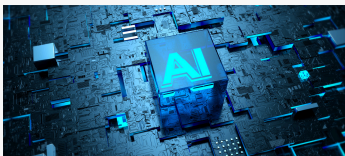
• 결론 및 향후전망

AI Driven Development에 따른 개발 환경 변화는 군 소프트웨어 개발에도 큰 영향을 미칠 것으로 예상된다. AI Driven Development의 발전은 군의 국방 소프트웨어 개발 시 현재의 획일적이고 관습적인 개발 방법론에서 벗어나 보다 정확하고 효율적이며 신속한 개발에 도움을 줄 것이다.

민간 기업과 달리 한정된 예산과 인력, 높은 요구 성능, 기존 체계와의 연동성을 고려해야만 하는 군 특성 상 AI의 도움을 받는 소프트웨어 개발 환경을 구축하여 각 군별 혹은 각 직능별, 무기체계별 요구되는 다양한 소프트웨어 개발 소요에 효과적이고 경제적이며 안정적으로 대응할 수 있도록 준비가 필요하겠다.

인공지능 기술의 개발은 그 무한한 가능성으로 인해 지속 발전할 것으로 보인다. 이에 발맞춰 AI Driven Development 역시 지금과는 다른 소프트웨어 개발 환경을 만들어 줄 것이며, 소프트웨어의 질적·양적 발전 역시 현재까지와는 차원이 다른 속도를 보여 줄 것으로 판단된다. 점점 더 첨단화되는 국방 분야, 특히 무기체계 개발 분야에서 AI Driven Development의 필요성은 필수적일 것이며, 우리 군은 이러한 신기술을 적용하고 발전시키는 데 적극적으로 임해야 할 것이다.

대표 이미지



원고 첫머리에 넣어 주세요.jpg

목록

댓글
0 / 500

로그인

최신순 | 추천순

<< < > >>

- | | | | | | |
|---|--|----|--|----|----------------------------------|
| 1 | LIG넥스원, 쉴드 AI 무인기에 ‘공대지 유도탄’ 장착한다 | 6 | 현대로템, 중동 겨냥한 ‘K2ME’ 전차 첫 출하…방위사업법 개정 결실 | 11 | 한화시스템, 새로운 ‘천공의 눈’ 만든다…천공-III |
| 2 | 해군 첫 3600톤급 ‘장영실함’ 진수…최첨단 기술… | 2 | | 7 | [최신 무기의 세계] K방산 독주에 전차군단 130mm … |
| | | 2 | 댓글 | 4 | 댓글 |
| 3 | LIG 디앤에이, 유도로켓 ‘비궁’ 美 수출 위한 교두보로 현지법인 설립 | 8 | [2026 국군 무기도감] 업그레이드 참수리 'R' 추가 … 압도적 화력 … | 13 | ‘해병의 첫 코뿔소’ 현대.템, 장애물개척전차 해.. |
| 4 | 록히드마틴, 2025년 F-35 191대 인도…사상 최대 기록 | 9 | [최신 무기의 세계] 세계 첫 40mm 기관포 탑재 막강 … | 1 | 댓글 |
| 5 | HD현대, 캐나다 잠수함 수주에 수조원대 ‘대규모 패키지딜’ 제안 | 10 | 한화, 한국판 ‘그레이 이글’ 무인기 만든다…美 GA와 국내 생산시설 … | 15 | 대한민국 해군 문무대왕국 250주년 기념 국제관 |

< 1 / 3 >

커뮤니티 인기 게시물

- 1 [BEMIL 현장취재] KF-21, F-35 등 최첨단 전력 집결한 서울 ADEX 2025 야외전시 폴사진



- 2 KF21시제기 가 인도네시아로 보내진다고 합니다.

- 3 미 중부사령부, GBU-72로 이란 지하 미사일 기지 두 곳 정밀 타격

- 4 트럼프, 한국 핵잠 제조 승인.

- 5 한화의 미래잠수함이 영국의 차기잠수함들과 외형이 흡사하지 않나요?

- 6 Accidents during the battle of Chinese-made VT-4 tanks by the Royal Thai Army

- 7 탐건의 고향인 San Diego MCAS Miramar 에서 촬영한 America's Airshow 사진들



- 8 KF-21EX, 스텔스에 더해 2000파운드 폭탄 2발 내부무장창에 장착 가능



- 9 중국 항모들의 근황

BEMIL 뉴스

BEMIL 포커스

커뮤니티

고객센터

제출기간 : 2019. 11. 15. ~ 2019. 12. 15. | 대상분야 : 사회과학 | 연구기간 : 2020. 1. 1. ~ 2020. 12. 31.